

Expt. No. : 7

Date :

SHAPE, RECTANGLE AND CIRCLE CLASSES

AIM

To write a python program to create a base and derived class and inherit the properties of base class to derived class.

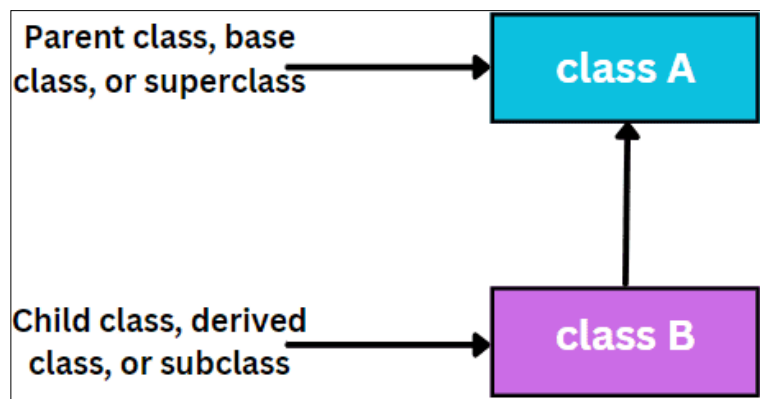
SOFTWARE REQUIRED

Python 3.14

THEORY

SINGLE INHERITANCE

In python single inheritance, a derived class is derived only from a single parent class and allows the class to derive behaviour and properties from a single base class. This enables code reusability of a parent class, and adding new features to a class makes code more readable, elegant and less redundant. And thus, single inheritance is much safer than multiple inheritances if it's done in the right way and enables derived class to call parent class method and also to override parent class's existing methods



Syntax

```
class Parent_class_Name:
```

```
    #Parent_class code block
```

```
class Child_class_Name(Parent_class_name):
```

```
    #Child_class code block
```

PROBLEM

1. Build a Python program to create a base class called Transformer that has a method called voltage_ratio() which returns 0. Then, create two derived classes StepUpTransformer and StepDownTransformer that inherit from the Transformer class to calculate the voltage ratio.

2. A power generation station records the operating details of generators. Build a Python program to create a class called Generator with attributes generator_name, rated_power, and operating_hours. Implement a method that returns the total electrical energy generated.

3. You are given four points A, B, C and D in a 3-dimensional Cartesian coordinate system. You are required to print the angle between the plane made by the points A, B, C and B, C, D in degrees (not radians). Let the angle be PHI.
 $\cos(\text{PHI}) = \frac{(X \cdot Y)}{|X| |Y|}$ where $X = AB \times BC$ and $Y = BC \times CD$.
 Here, $(X \cdot Y)$ means the dot product of X and Y, and $AB \times BC$ means the cross product of vectors AB and BC. Also, $AB = B - A$.

ALGORITHM

STEP 1: Start the Program

STEP 2: Input Length and Width

- Prompt the user to enter the length of the rectangle.
- Prompt the user to enter the width of the rectangle.
- Read the user input and convert it to floating-point numbers.

STEP 3: Create Rectangle Object

- Initialize an instance of the Rectangle class with the provided length and width.

STEP 4: Calculate Area

- Call the area method on the Rectangle object to compute the area.

STEP 5: Display Result

- Print the calculated area of the rectangle.

STEP 6: Handle Invalid Input

- If the user input is not a valid number, display an error message indicating invalid input.

STEP 7: End the Program

PROGRAM CODING

```

class Transformer:
    def voltage_ratio(self):
        return 0
class StepUpTransformer(Transformer):
    def __init__(self, primary, secondary):
        self.primary = primary
        self.secondary = secondary
    def voltage_ratio(self):
        return self.secondary / self.primary
class StepDownTransformer(Transformer):
    def __init__(self, primary, secondary):
        self.primary = primary
        self.secondary = secondary
    def voltage_ratio(self):
        return self.secondary / self.primary
vp = float(input("Enter Primary Voltage/Turns: "))
vs = float(input("Enter Secondary Voltage/Turns: "))
if vs > vp:
    t = StepUpTransformer(vp, vs)
    print("Transformer Type: Step-Up Transformer")
else:
    t = StepDownTransformer(vp, vs)
    print("Transformer Type: Step-Down Transformer")
print("Voltage Ratio:", round(t.voltage_ratio(), 2))

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment7.py =====
Enter Primary Voltage/Turns: 200
Enter Secondary Voltage/Turns: 100
Transformer Type: Step-Down Transformer
Voltage Ratio: 0.5
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment7.py =====
Enter Primary Voltage/Turns: 100
Enter Secondary Voltage/Turns: 200
Transformer Type: Step-Up Transformer
Voltage Ratio: 2.0
>>>

```

PROBLEM CODING

```

class Generator:
    def __init__(self, name, power, hours):
        self.generator_name = name
        self.rated_power = power
        self.operating_hours = hours
    def total_energy(self):
        return self.rated_power * self.operating_hours
name = input("Enter Generator Name: ")
power = float(input("Enter Rated Power (in kW): "))
hours = float(input("Enter Operating Hours: "))
gen = Generator(name, power, hours)
print(f"Total Energy Generated by {gen.generator_name}: {gen.total_energy()} kWh")

```

OUTCOMES OF PROGRAM CODING

```

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem\experiment7.py =====
Enter Generator Name: GEN-1
Enter Rated Power (in kW): 50
Enter Operating Hours: 8
Total Energy Generated by GEN-1: 400.0 kWh
>>>

```

PROBLEM CODING

```

import math
import numpy as np
A = np.array(list(map(float, input("Enter A: ").split()))))
B = np.array(list(map(float, input("Enter B: ").split()))))
C = np.array(list(map(float, input("Enter C: ").split()))))
D = np.array(list(map(float, input("Enter D: ").split()))))
AB = B - A
BC = C - B
CD = D - C
X = np.cross(AB, BC)
Y = np.cross(BC, CD)
cos_phi = np.dot(X, Y) / (np.linalg.norm(X) *
np.linalg.norm(Y))
phi = math.degrees(math.acos(cos_phi))
print("Angle =", round(phi, 2))

```

OUTCOMES OF PROGRAM CODING

```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem1\7.py =====
Enter A: 0 0 0
Enter B: 1 0 0
Enter C: 1 1 0
Enter D: 1 1 1
Angle = 90.0
>>>

```

VIVA QUESTION**1. What is inheritance in Python, and why is it used?**

Inheritance is a mechanism that allows a class (derived class) to inherit attributes and methods from another class (base class). It promotes code reusability and establishes a relationship between classes.

2. How do you define a base class and a derived class in Python?

```
class BaseClass:
    pass
class DerivedClass(BaseClass):
    pass
```

3. How do you call a method from a base class within a derived class?

```
class BaseClass:
    def method(self):
        print("Base class method")

class DerivedClass(BaseClass):
    def call_base_method(self):
        self.method() # Calls the base class method
```

4. What does the super() function do, and how is it used in Python?

```
class BaseClass:
    def __init__(self):
        print("Base class constructor")

class DerivedClass(BaseClass):
    def __init__(self):
        super().__init__() # Calls the base class constructor
        print("Derived class constructor")
```

5. How can you prevent a method in a derived class from overriding a method in the base class?

You can use a naming convention (prefixing with _) or raise an exception in the base class method to indicate it should not be overridden.

```
class BaseClass:
    def method(self):
        raise NotImplementedError("This method should not be overridden")
```

6. What is the difference between single inheritance and multiple inheritance in Python?

Single inheritance involves one base class and one derived class, while multiple inheritance involves a derived class inheriting from more than one base class.

```
class Base1:
    pass
class Base2:
    pass
```

```
class Derived(Base1, Base2): # Multiple inheritance
    pass
```

7. How do you initialize the base class in the constructor of a derived class?

```
class BaseClass:
    def __init__(self):
        print("Base class initialized")
class DerivedClass(BaseClass):
    def __init__(self):
        super().__init__() # Initialize base class
        print("Derived class initialized")
```

8. What is method overriding, and how is it achieved in Python?

Method overriding occurs when a derived class provides a specific implementation of a method that is already defined in its base class.

```
class BaseClass:
    def method(self):
        print("Base class method")
class DerivedClass(BaseClass):
    def method(self): # Overrides the base class method
        print("Derived class method")
```

9. How does Python handle method resolution order (MRO) in multiple inheritance scenarios?

Python uses the C3 linearization algorithm to determine the method resolution order in multiple inheritance scenarios, ensuring a consistent order for method lookup.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
print(D.mro()) # Output: [<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```

10. What are some potential issues that can arise from using multiple inheritance, and how can they be managed?

Potential issues include the diamond problem (ambiguous method resolution). These can be managed by using the `super()` function correctly and being aware of the MRO.

RESULT:

Thus, the Python program successfully created a base class named Shape and derived classes Rectangle and Circle using inheritance. The overridden area() methods correctly calculated the areas of the respective shapes based on user input. The program was executed successfully, and the output was verified.

INFERENCE:

The experiment demonstrates that inheritance enables derived classes to reuse the properties of a base class while implementing their own specialized behaviour through method overriding. The successful execution of the program validates the concepts of inheritance, polymorphism, class creation, and object instantiation in Python, making the code modular, reusable, and easier to maintain.

Assessment	Max. Marks	Marks Obtained
Conduction of Practical	15	
Report	5	
Viva	5	
Total Marks	25	
Signature of Lab In-Charge		

